

The final feature of C we need is the comment: anything enclosed by `/*` and `*/`. This is useful, in writing "pigeon C" programs, to specify an operation. For example, we can write a specification for a program that, when getting a key command from a keyboard, and executes a "go" operation if the key is 'g', a reverse operation if 'r', and a stop operation if 's':

```
main()
{
    char keycode;
    /* get a key into keycode */
    switch (keycode) {
        case 'g': /* go */ break;
        case 'r': /* reverse */ break;
        case 's': /* stop */ break;
    }
}
```

The flow chart for this program is shown in figure 2-3b. A "pigeon C" program can also call procedures that are themselves not written yet, but are clearly understood.

When an assembler language program is written, a flow chart should be given (unless it is trivial), and the equivalent C program should be shown. Other comments should be added to explain special requirements or "tricky" operations. In particular, subroutines should be extensively commented, as we discuss later. This section provides enough information about C so you can use it to document assembler language programs.

2-2.2 Programming Style

We conform our programming techniques to some style to make the program easier to read, debug, and maintain. Although this constraint generally produces less efficient code, the use of a consistent style is recommended, especially in longer programs where there is enough memory, but not where static efficiency is paramount. One programming style - discussed first - is to rigidly enforce reentrancy and use some conventions to make this rather automatic. The second programming style we discuss - structured programming - uses only simple conditional and loop operations and avoids GOTO statements. After this we discuss top-down and bottom-up programming. The final style we describe is the rigid enforcement of position independence, a style easier to enforce in more powerful architectures like the Motorola 6809 and 68000 but which can be enforced with some loss of efficiency in the 6811.

Program segments may have multiple entry points and multiple exit points. (See figure 2-5.) Such a program segment occurs anywhere in a program, usually a part written on consecutive lines. An *entry point* is any instruction that may be executed after an instruction not in the program segment, and an *exit point* is any instruction that may be followed by an instruction not in the program segment. A program segment may or may not be a subroutine; when it is, the discussion here also applies to the next section on subroutines.

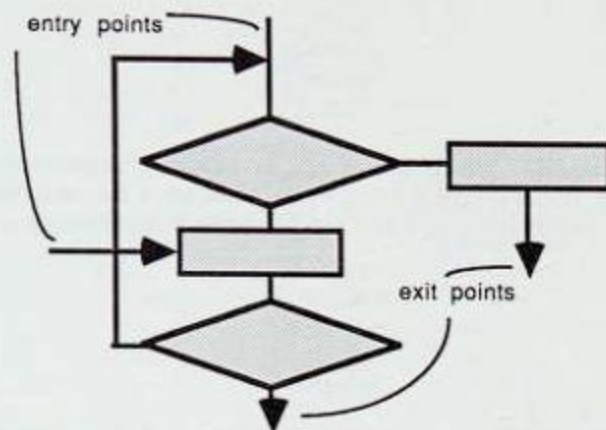


Figure 2-5. A Program Segment

The main problem in writing good programs is the handling of variables. Variables used in programs are local or global. *Local variables* are temporary variables used only by one program segment. *Global variables* are variables shared by two or more program segments. For example, several program segments may be written to handle a floppy disc, and global variables may be used to keep track of the floppy disc's status, so all program segments can access those variables. Only variables shared by several routines, which are to be located independent of the order of calling subroutines, are global variables.

In the 6811, local variables should be stored on the stack, for reasons which will be given at the end of this discussion. In the 6811, if global variables are short, they should be stored on the direct page, or if they would use too much of the direct page, their address could be stored on the direct page to be loaded into an index register efficiently. However, the ability to use index addressing on the stack in the 6811 encourages the use, whenever possible, of dynamic local variables rather than global variables.

A convention that greatly aids a program's clarity is this:

1. Determine what a program segment will be. Usually it is a subroutine.
2. Determine what variables in the program segment are local. These are temporary variables used in the program segment and not anywhere else.
3. On each entry point to the program segment, allocate *all* the local variables at once and in the same way on each entry point; and on each exit point, deallocate *all* the local variables in the same way, as shown next. Do *not* push or pull anything on the stack in the rest of the program segment (only nesting is permitted, as discussed later).

To allocate local variables on the stack, we move the stack pointer towards lower memory. We can make room for *n* variables at once by the sequence:


```
TSX
XGDX
SUBD #n
XGDX
TXS
```

Of course, for less than 7 bytes of local storage, use as many DES instructions as needed. In that case, use TSX at the end to make X point to the top of the stack, for reasons that appear shortly. We can deallocate n local variables by the instruction sequence

```
TSX
XGDX
ADDD #n
XGDX
TXS
```

Of course, for less than 7 bytes of local storage, use as many INS instructions as needed.

If the allocation just described is done on each entry point, then X points to the top of the stack, and index addressing using X can access the local variables. Index register Y is available for handling data structures. If both index registers are needed, X can be easily restored to its position by TSX after its use for some other operation. In the preceding discussion, index register Y can be exchanged with X if that improves efficiency. For example, consider a procedure with some local variables

```
sub()
{
    char j,k,l,m;
    j=k+l+m;
}
```

It is coded as:

```
SUB    DES    ALLOCATE
        DES    ALL FOUR
        DES    LOCAL
        DES    VARIABLES
        TSX    SET X
        LDAA  1,X  GET K
        ADDA  2,X  ADD L
        ADDA  3,X  ADD M
        STAA  0,X  PUT IN J
        INS    DEALLOCATE
        INS    ALL FOUR
        INS    LOCAL
        INS    VARIABLES
        RTS
```

Note that the declaration of variables in C directly corresponds to the allocation of variables in assembler language. Use equates to make the code more readable and to avoid bugs:

```
J      EQU    0
K      EQU    1
L      EQU    2
M      EQU    3
SUB    DES    ALLOCATE
        DES    ALL FOUR
        DES    LOCAL
        DES    VARIABLES
        TSX    SET X
        LDAA  K,X  GET K
        ADDA  L,X  ADD L
        ADDA  M,X  ADD M
        STAA  J,X  PUT IN J
        INS    DEALLOCATE
        INS    ALL FOUR
        INS    LOCAL
        INS    VARIABLES
        RTS
```

In the program segment, the local variables can each be *accessed using the same offset* from the X pointer (which points to the top of the stack) to get the local variables on the stack every time the variable is used, and S may be transferred to X any time X has to be restored, because we do not push or pull on the stack. Equates can be used to make this quite readable.

Normally, local variables are undefined when the program segment begins execution, and may have to be *initialized* before they have known values. That is, values may have to be put in the memory locations allocated for the variables. This is especially true of reentrant subroutines, and forgetting to initialize them is a common and insidious bug. Note that PSHA and similar instructions can simultaneously allocate and initialize the local variables for better efficiency.

Variables have to be deallocated at the end of the program segment. Note that deallocation can be done by PULA and similar instructions if the local variable is needed, such as when it is the return value of a function. Deallocation makes the physical memory available for another program segment so the same memory gets used over and over again. Memory is thus optimally used: the amount a program needs is just the sum of the number of local variables of the worst case nesting of program segments (including subroutines), where the worst case is a point in running the program where the largest number of local variables are stacked up.

In some block-structured languages like ALGOL and PASCAL the program blocks are written so the inner, lower-level blocks can access local variables of outer blocks, using the dynamic (run-time) calling sequence. This can also be done in assembler language programs. Within inner program segments, you can access local variables (called *dynamic local variables*) of outer program segments, if you know where they are located. You can locate them by knowing how many additional bytes have been allocated on the stack since the outer segment allocated the variable you access in the inner segment

and adding this number to the offset you used, thus getting the variable in the outer program segment. Figure 2-6 shows the main idea. The local variable of an outer segment is accessed in an inner segment by using an offset equal to the offset used in the outer segment plus the number of bytes that have been allocated since the outer segment. (We assume the inner segment is not a subroutine, so there is no return address on the stack.) Of course, equates can be used to clarify the program. For instance, if the variable is ALPHA in the outer segment, we write ALPHA EQU 0, and the first access to ALPHA in the outer segment can be written STAA ALPHA,X. The directive SGALCT EQU 2 can be put at the beginning of the inner segment, and the second access to ALPHA in the outer segment can be written LDAA ALPHA+SGALCT,X. After the inner segment is over, the third access to ALPHA in the outer segment is written LDAA ALPHA,X.

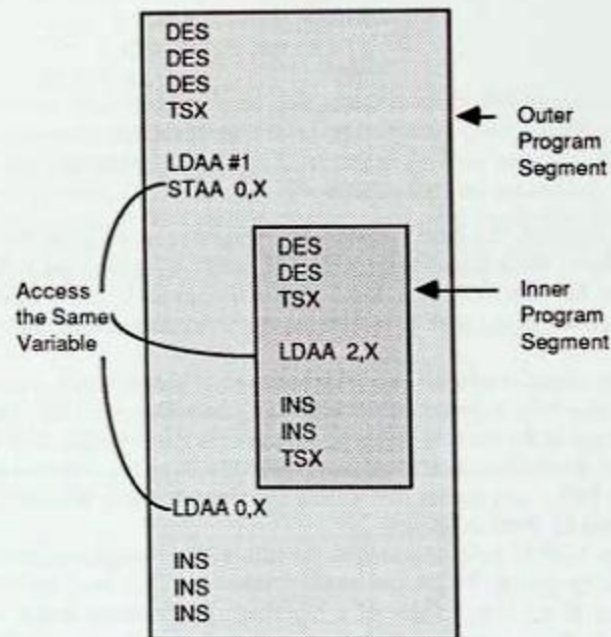


Figure 2-6. Nested Program Segments

One can use inner segments to pass arguments to subroutines using the stack. One can also use the stack as a very temporary variable, although this use is almost trivial. For example, the best way to transfer X to D is the sequence

```
PSHX
PULA
PULB
```

One can identify the allocation, initialization, and deallocation of this program segment, which is an inner program segment of a larger one, but it is rather trivial because no local variables are accessed in it, so we won't bother to make a big issue of it.

In the 6811, the use of the stack to store local variables is usually recommended. (With 8K words of ROM, there is usually enough room for extra instructions.) This technique optimally uses the scarce 256 words of RAM. Also, the programs become reentrant, so they can – as all such programs can – be used in interrupt handlers. You may never know when a routine may have to be used in a handler, so if it is already reentrant, you won't need to rewrite it. For example, we once wrote a text editor in which all the code was in some interrupt handler or another and the code in the main program did nothing. Also, you may use a routine that is not reentrant in a handler and find out that the resulting program works most of the time but fails sometime for no apparent reason – because an interrupt happened to occur while a non-reentrant program segment, being used by the handler, was being executed.

We now turn our attention to another programming style that can be used in microcomputers to improve the code's clarity. An element of *structured programming* is the use of single entry point, single exit point program segments. This style makes the program much more readable because, to get into a program segment, there are no circuitous routes which are hard to debug and test. The use of C for specification and documentation can force the use of this style. The conditional and loop statements described in the last section are single entry point, single exit point program segments and they are sufficient for almost all programs. The *while* loop technique is especially attractive because it tests the termination condition before the loop is done even once, so programs can be written that accommodate all possibilities, including doing the loop no times. And the *for* loop is essentially a beefed-up *while* loop. You can use just these constructs. That means avoiding the use of GOTO statements. Several years ago, Professor Edsger Dijkstra made the then controversial remark, "GOTOs Considered Harmful!" Now, most good programmers agree with him. The only significant exception is the reporting of errors. We sometimes GOTO an error reporting or correcting routine if an error is detected – an abnormal exit point for the program segment. We heard a story (from Harold Stone) that Professor Goto in Japan has a sign on his desk that says "Dijkstras Considered Harmful!" (Professor Goto denies this.) Errors can alternatively be reported by a convention such as using the carry bit to indicate the error status: you exit the segment with carry clear if no errors are found and exit with carry set if an error is found. Thus, all segments can have single exit points.

Top-down design is a program-writing methodology that has produced programs more quickly than adhoc and haphazard writing. The idea is to write a main program that calls subroutines (or just program segments) without yet writing the subroutines (or segments). A subroutine is used if a part of the program is called many times, and a program segment – not a subroutine – is used if a part of the program is used only once. The main program is executed to check that the subroutines and segments are called up in the proper order under all conditions. Then the subroutines (or segments) are written in lower level subroutines (or segments) which are then tested. This is continued until the lowest level subroutines (or segments) are written and tested. This methodology requires the use of superior documentation to describe the subroutine and program segments so they can be fully tested before they are actually written. "Pigeon C" is a very useful tool for that purpose. Also, the inputs and outputs of subroutines have to be carefully specified. This will be discussed at the end of the next section.

The inverse of top-down design is bottom-up design, in which the lowest level subroutines or program segments are written first and then fully debugged. These are built up, bottom to top, to form the main program. To test the subroutine, you write a short program to call the subroutine, expecting to discard this program when the next higher level program is written. Bottom-up design is especially useful in interface design. The lowest level subroutine which actually interfaces to the hardware is usually the trickiest to debug. This methodology lets you debug that part of the program with less interference from other parts. Bottom-up design is like solving an algebra problem with three separate equations in three separate unknowns, each equation being in one unknown. Whereas, putting all the software and hardware together before testing any part of it is like simultaneously solving three equations in three unknowns. As the first algebraic problem is much easier, the use of bottom-up design is also a much easier way to debug interfacing software. In a senior level interfacing course at the University of Texas, students who tried to get everything working at once spent 30 hours a week in lab, while those who used bottom-up design spent less than 10 hours a week on the same experiments.

Combinations of top-down and bottom-up design can be used. Top-down design works well with parts of the program that do not involve interfacing to hardware, and bottom-up design works better with parts that do involve interfacing.

Position independence can be enforced in the 6811, although that is not always desirable. The BSR, BRA, and conditional branch instructions provide for position independence for short programs. The long jumps, however, can be position independent using the following code in place of a jump such as JMP L2:

```

        BSR    L1
L1      PULA
        PULB
        ADDD   #L2-L1
        PSHB
        PSHA
        RTS

```

Note that BSR is used as a kind of push PC, and RTS as a kind of pull PC instruction. The ADDD instruction adds the position independent displacement to the program counter. A similar segment may be used for jumps to subroutines. This kind of routine should only be used if the labels L1 and L2 are moved together as the program is moved around, such as when they are both in the same external ROM in an extended multiplexed bus mode MC68HC11. This provides the capability to write ROM that can be used in different systems, although they may appear in different places in each system's memory map. Because the same ROM can be used in more systems, its design cost is spread out over more users, and thus its cost is less. A ROM similar to the Motorola 6839, which is used in 6809 systems to provide floating point subroutines, could be made for the 6811; such a ROM should use position independent code.

In this section, we discussed some programming styles useful in larger programs on the 6811. These styles do not produce the most statically and dynamically efficient code, so they may not be useful if efficiency is required. However, the use of local

variables on the stack to enforce reentrancy and conserve RAM memory is often desirable; the use of structured programming - especially single entry, single exit program segments - and the use of top-down and bottom-up design are often desirable; and the enforcement of position independence is occasionally necessary.

2-3 Subroutines and Arguments

One area, often inadequately understood, of great importance in microcomputer interfacing software is the way subroutines are called, and the way arguments are passed into them and results are passed from them. Since the instruction set of a typical microcomputer is rather limited, the kinds of functions usually needed in most applications are implemented as subroutines. In fact, the code generated by some high-level language compilers consists usually almost exclusively of subroutine calls, as we noted earlier. These techniques for programming subroutines are discussed in this section.

The passing of arguments has two aspects. A conceptual or strategic aspect is associated with how you want the argument treated, and an implementation or tactical aspect is associated with how you actually do it. The conceptual aspect is often covered in computer science courses. It is especially important in high-level languages, because when you program in these languages you really don't care how the arguments are actually handled, provided they are handled in a conceptually consistent manner. Even in assembler language, the conceptual aspect is important because you want to know what you are doing in a strategic sense. We cover this first. The implementation aspect, discussed next, is also important to assembler language programmers because they must be concerned with how the arguments are passed. We then discuss the calling and returning mechanisms to show alternatives to the BSR and RTS instructions. Finally, we discuss the subroutine documentation and give a concise blueprint for writing subroutines with style.

2-3.1 Passing Parameters (Conceptual Level)

Conceptually, *arguments* or *parameters* are data passed from or to the calling routine to or from a subroutine, like the x in $\sin(x)$. In an implementation, the parameter has either some register in the machine state or some storage location, with a symbolic address associated with that location inside the subroutine, such as y . It is called the *formal parameter*. The calling routine has a register or memory location with its symbolic address, available for the parameter that is usually different each place the subroutine is called. The register or memory location with its symbolic address in the calling routine is called the *actual parameter*. For example, at one place in the program we put $\sin(\alpha)$, in another, $\sin(\beta)$, and in another, $\sin(\gamma)$. α , β , and γ are actual parameters in these examples.

At the conceptual level, arguments are called by value and result, reference, or name. In *call by value and result*, the formal parameters inside the subroutine are usually registers. The values of the actual parameters from the calling routine are really transferred from their memory locations in the calling routine to the registers in the subroutine before the subroutine begins to execute its operation. The values of formal parameters in the registers are moved to their actual parameters after the subroutine is

finished. However, any mechanism whereby the actual values (rather than their addresses) are passed to the subroutine or actual results (rather than their addresses) are returned from the subroutine are called by value and result. In general, only parameters passed from the calling routine to the subroutine are moved before the subroutine begins execution, and only the results of the subroutine are copied into the calling routine's actual parameters after the end of a subroutine's execution, but the same formal parameter (register) can be used for input and output.

In *call by reference*, the data remains in the calling routine and is not actually moved to another location, but the address of the data is given to the subroutine and the subroutine uses this address to get the data whenever it needs to. Large vectors, lists, arrays, and other data structures can be more effectively called by reference so they don't have to be copied into and out of the subroutine's local variables. Conceptually, arguments passed in call by reference are evaluated just before the subroutine is called and are not reevaluated as the subroutine is executed. If an argument is called by reference, you should normally use it only for input, or only for output, and not both, since input arguments are supposed to behave as if they were evaluated just before the subroutine call, and are supposed to stay that way throughout the subroutine.

The last mechanism, *call by name*, allows complex actual arguments to be passed to a subroutine. These actual parameters are effectively (if not actually) inserted into every occurrence of the corresponding formal parameters inside the subroutine and are reevaluated each time they are met as the subroutine is executed. Call by name is useful when you want to refer to an argument by its address but you change it in the subroutine, so it has different values at different times in the subroutine. Call by name is also useful when actual parameters are subroutines, for example, as arguments to another subroutine. If you wrote a subroutine called *plot* to plot a graph, you could pass an argument that would be the address of a subroutine like *sin* - as in *plot(&sin)* - and *sin* would be reevaluated each time the *plot* routine was ready to plot a new point on the graph. If you used call by reference, the argument *sin* would be evaluated just once, just before the subroutine was entered, so your plot would be a straight line. Finally, conceptually, call by name is used to handle error conditions. One argument - the address of a subroutine to go to if an error is detected - is only executed if the error occurs and so is a call by name argument.

In C, the *&* and *** operators are used to indicate call by name. Otherwise, (scalar) parameters are passed in call by value, and the result of a function is passed in call by result. Parameters inside the parentheses can only be passed into the procedure and not out of it. Suppose a procedure is used to evaluate the power of a number instead of a function, as was used earlier. We can use call by name to get the result of the operation out of the procedure. The procedure is written

```
power(i,j,n)
int i,j,*n;
{
    int k;
    n=1;
    for(k=0;k<j;k++) n=n*i;
}
```

and the calling routine calls up such a procedure with *power(i,3,&r)*. In the procedure, the result is put in the last parameter. As parameters inside the parentheses are only put into the procedure, we pass the address of the result into the procedure (hence the *&* in front of the actual parameter *r*). Inside the procedure, when we declare the third formal parameter, we use *** to say that the word *n* points to is an integer. The C compiler generates the correct code for using the address to get the value out of the procedure. Using call by name, C can pass the address of a parameter into the procedure, so data can be returned from the procedure using that address.

2-3.2 Passing Parameters (Implementation Level)

We now consider implementation details of the 6811's subroutine call arguments. The passing of arguments is implemented by means of registers, global variables, a stack, an argument list, or a table. In discussing these techniques, we'll be concerned with 6811 features useful in carrying them out.

In the 6811, the X,Y, and D (or A and B) registers can serve to hold arguments or results. While the index registers naturally lend themselves to passing addresses - in call by reference or call by name - these can be transferred to or exchanged with the D register using XGDX or XGDY instructions, so they can also be used to pass parameters by value. Also, the carry bit in the condition code register can be used to pass a 1-bit result that can be used in instructions like BCC and BCS. If a subroutine does not have many arguments, this is usually the best way, being easy to understand and use, to pass them. Its main disadvantage is that it is not completely general. Most high-level languages need a completely general technique because compilers are usually not smart enough to pass parameters some ways to some subroutines and other ways to other subroutines. Therefore, if you want to use a subroutine that was written to be called by a high-level language routine or you want to write a subroutine that can be called by a high-level language routine, you may have to pass the arguments in one of the ways described next.

In an example of passing arguments by registers, assume all variables are of type *char* and the calling routine is *m=power(h,3);*. The subroutine is described in C as

```
power(i,j)
char i,j;
{
    char n;
    n=1;
    for( j>0 ;j-- ) n=n*i;
    return(n);
}
```

This is coded in assembler - first the calling routine, then the subroutine - as follows

```
LDAA H
LDAB #3
BSR POWER      m=power(h,3);
STAB M
```


POWER	PSHA	<i>power(i,</i>
	PSHB	<i>j) char i,j;</i>
	TSY	
	LDAB #1	<i>char n; n=1</i>
L1	DEC 0,Y	<i>for(; j<0;j--)</i>
	BMI L2	
	LDA 1,Y	<i>n=n*i;</i>
	MUL	
	BRA L1	
L2	INS	BALANCE THE STACK
	INS	
	RTS	<i>return(n);</i>

An argument can be put in a global variable, and then the subroutine can find the argument there. Results can be passed in the other direction through global variables. This is like supplying arguments through FORTRAN COMMON. It is too easy to forget to supply one of the critical arguments, and it is easy to use the same location inadvertently to supply an argument to a subroutine that calls another, and for that subroutine to supply an argument to the subroutine that it calls. Also, arguments for one subroutine can accidentally get mixed up with arguments in another subroutine. To diagram such a situation, we use *coat hanger diagrams*. (See figure 2-7.) Each horizontal row represents the execution of a routine or subroutine, and the calling and returning points are shown as breaks in the line. If we use a global variable like ARG1 in the first subroutine after another subroutine (perhaps several levels down) that also uses the variable ARG1 is called, then the first subroutine will have the wrong value in ARG1 when it uses ARG1. Finally, such a technique is not reentrant. Global variables have to be used to pass arguments in very small computers like the 6805. There is no way to get to the stack or the return address to use the techniques described next. However, if either of these paths is available in the architecture, global variables should be avoided. We recommend against passing arguments in global variables.

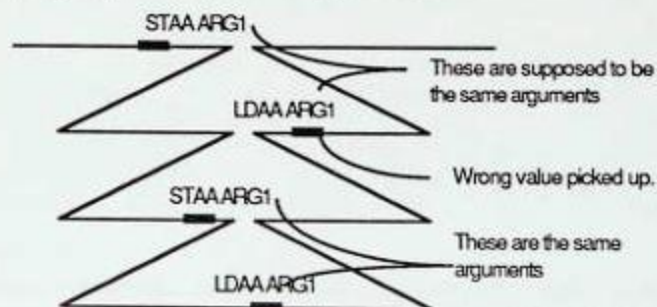


Figure 2-7. Coat Hanger Diagram Showing a Bug Using Global Variables

The stack provides a completely general mechanism for passing arguments. Suppose the argument X for the subroutine SIN(X) is in accumulator A. To call the subroutine SIN, execute the following program segment:

```
PSHA
BSR SIN
```

Inside the subroutine, we can get the value of the argument, but the obvious way will not work. Do *not* execute the instruction

```
PULA
```

This will pull the high byte of the return address into A, rather than the argument you want. However, because the argument is the third word from the top of the stack, it can be picked up as follows:

```
TSX
LDA 2,X
```

When the subroutine is finished, the RTS instruction is used to pull the 2-word return address from the stack. But we are not through. The argument is still on top of the stack. If the calling routine is itself a subroutine, when it executes its RTS instruction, it will pull the argument from the stack into the program counter and jump to some dangerous place. After the subroutine has returned to the calling routine, the calling routine must remove the argument (balance the stack). To delete it, you could use a PULA instruction, or, if the argument is no longer needed, an INS instruction. You should recognize this as the special nested program segment case discussed in the last section, whereby the subroutine is (dynamically) executed within the small program segment that puts the arguments on the stack for the subroutine, using the same rules to find variables on the stack for subroutines as were used in inner and outer program segments. You can also think of this as providing "holes on the stack" through which to pass the arguments.

Any number of arguments can be pushed onto the stack this way, and can, if the instruction TSX is used at the beginning of the subroutine, be accessed inside the subroutine using index addressing with the X register. Values can be passed in the manner described, as can addresses be passed for call by reference, or subroutine addresses for call by name. As an example of the latter, the *plot* subroutine may need an argument which is the subroutine of the function to be plotted, such as the SIN routine. To pass the address of the *sin* subroutine, we can execute this routine:

```
LDX #SIN
PSHX
BSR PLOT
```

(The preceding example is not position independent, but can be made so.) Inside the *plot* subroutine, to call the *sin* subroutine, the instruction

```
TSX
LDX 2,X
JSR 0,X
```


can be used. Note that after the *plot* subroutine returns to the calling routine, the argument must be deleted from the stack so two *INS* instructions can be used.

Results can also be returned on the stack, but again the obvious way doesn't work. If, inside the subroutine, you push a result on the stack, then return and expect to pull it from the stack, the return instruction *RTS* will pull the result before you can, which will effect a jump to some unknown place. Rather, a "hole" for the result is inserted on the stack before the subroutine is called; the result can be put in the hole with a technique that is the reverse of the one used to pass arguments to the subroutine. Again, this is just a special case of the inner and outer program segment access mechanism described in the last section. The calling routine can be written

```
DES
BSR SIN
PULA
```

And the subroutine can put the result in the "hole" by executing the instructions

```
TSX
STAA 2,X
```

Note that any number of results can be passed this way. Before the subroutine is called, a "hole" must be created for each result, and, to balance the stack after the subroutine has returned to the calling routine, each result must be pulled from the stack (or deleted). Thus, the stack is a completely general technique for passing arguments and results.

If a second stack is used, not the one used to save the return address, then arguments can be pushed on that stack by the calling routine and pulled by the subroutine, without fear of messing up the return addresses or having them in the way. This technique is especially useful for arithmetic routines and will be used in section 2-4.

The next technique for passing arguments is the *argument list*. It is almost as general as the stack technique, and is the most commonly used for high-level language subroutines. A high-level language subroutine call, like the *FORTRAN* statement

```
CALL SUB (A,B,C)
```

is most efficiently implemented (using call by reference) in assembler language as the following code:

```
BSR SUB
FDB A,B,C
```

Note that the microprocessor, when it executes the *BSR* instruction, saves what it thinks is the return address, which is actually the address of the word below the *BSR* instruction. That turns out to be fortuitous. It helps us get to the argument list, which is the list of addresses created by the *FDB* directive. To put the value of the word at address *C* into accumulator *B* inside the subroutine, execute the following program segment:

```
TSX
LDX 0,X    GET THE ADDRESS OF THE ARGUMENT LIST IN X
LDX 4,X    GET THE THIRD ARGUMENT INTO X
LDAB 0,X   GET THE VALUE OF C INTO ACCUMULATOR B
```

Note that once one of the index registers, like *X*, points to the argument list, any argument can easily be obtained by repeated use of index addressing. If *C* is the address of an array, that address can be put into an index register so index addressing can be used to access words in that array, as follows:

```
TSX
LDX 0,X    GET ADDRESS OF ARGUMENT LIST
LDX 4,X    GET ADDRESS OF ARRAY INTO X
```

Similarly, values can be passed and arguments can be called by name, through the argument list. Note that simple execution of *RTS* will not work, because the first argument will be executed as an instruction. Rather, if the argument list is 6 words long, we can execute the following program segment rather than the *RTS*:

```
PULX
JMP 6,X
```

What makes this technique only *almost* completely general is that, especially in microcomputers, the program may be stored in read-only memory so it doesn't have to be loaded each time power is turned on. This means that the argument list, in read-only memory, cannot be changed. So if arguments are to be called by value, they cannot be placed in the argument list unless they are the same each time the subroutine is called from that place. That condition sometimes occurs, and we'll discuss an example with the *SWI* "subroutine" call shortly. Although not completely general, it is faster than the stack approach because the calling routine does not have to set up and delete the arguments. It can also be used by subroutines called by high-level language programs. Moreover, it can pass the relative address of a local variable on the stack, as long as this address is added to the stack pointer value when it is used, thus giving a reentrant capability to the calling program that is absent if the address is a global address.

A final technique is to pass, using an index register, the address of a list or table into the subroutine. The contents of the list can be essentially the same as the argument list just described, but since the list need not be in ROM, its elements can be modified and replaced with the results before the subroutine is called. The code to do this is essentially the same as for in-line argument lists, except the address of the table must be loaded into the index register before the call is made to the subroutine, and the subroutine need not extract the return address from the stack, nor need it branch around the argument list. Passing the address of a list is often used in subroutines that handle disks, because the same list can be used as an argument for different disk handling subroutines that access the same disk and different lists can be passed through those subroutines to handle different disks. This technique is often used in operating systems for "low-level access" to disks and other similar complex I/O devices.

2-3.3 Calling and Returning Mechanisms

Subroutines are called by the JSR and BSR instructions and use the RTS instruction to return to the main program. In addition, you can use the PULX JMP 0,X instructions rather than RTS to get around argument lists, as just described. However, the SWI and RTI pair can be used to advantage in some cases.

The SWI is an alternative to the usual BSR subroutine call. The 6811's SWI instruction can be attractive because it saves all the registers. And, as we have discussed, you may want an argument list to follow the SWI instruction, where one of the arguments (the first) is a call by value argument that tells you exactly which routine to execute. The "subroutine call" is written this way:

```
SWI
FCB 0
```

Inside the SWI "subroutine," the (only) argument can be obtained by using the saved PC, which are the seventh and eighth words from the top of the stack. To put this argument into accumulator B, execute the following as the first instructions in the "subroutine":

```
L    TSX
      LDX 7,X
      LDAB 0,X
```

(The address L is put into locations \$FFF6 and \$FFF7, so that SWI will cause a "jump" to that instruction.) This argument can be used to jump to a number of routines by jumping through a *transfer vector* - a vector of addresses of starting locations of the different routines, such as:

```
TVEC FDB L0,L1,L2,L3,L4
```

To jump to the *i*th routine, where *i* is in accumulator B, double B and add it to the address of the vector, then jump indirectly, as in

```
ASLB
LDX #TVEC
ABX
LDX 0,X
JMP 0,X
```

(The preceding transfer vector and routine are not position independent, but it is not too hard to make them so.) Note that if the argument after the SWI instruction were the number 2, we would jump to L2, and so on. This particular argument is likely to be the same actual argument, whenever the subroutine is called from the same place in memory, so it is an example of a call by value that can be stored in a read-only memory argument list. Note that more arguments can be passed in an argument list to an SWI. Finally, note that the RTI instruction will return to execute the arguments as instructions. To

avoid that, modify the return address before the RTI so it skips over as many words as there are arguments. For instance, to skip over one argument, execute this routine to exit from the SWI:

```
TSX
INC 8,X
BNE L
INC 7,X
L    RTI
```

The SWI is especially useful for I/O software that you might write to interface your hardware. Suppose a bug in the I/O software is discovered after a lot of code has been written that uses it. Fixing that bug may change the sizes and entry points for many of the I/O subroutines. If the SWI "subroutine" technique is used with the transfer vector, only the transfer vector has to be modified, and the other software will still use the same argument. If a BSR subroutine were used, every address in an I/O subroutine call would have to be changed throughout all the programs. SWI "subroutines" are therefore often used for I/O. However, the key aspect of the SWI mechanism is indirect addressing. If you have a transfer vector, like TVEC in the preceding example, in a fixed place in memory, the instruction LDX TVEC+6 JSR 0,X would jump to a (JSR type) subroutine at location L3. If you rewrote those subroutines, you would only need to rewrite the transfer vector, and all the programs calling the subroutines would remain the same. This technique is especially useful when programs are also on different read-only memories. If all subroutine calls to subroutines on a read-only memory JSR indirectly through a transfer vector at the beginning of that read-only memory, then if that read-only memory is changed, the transfer vector can also be changed, and no other read-only memories need changing to adjust the addresses of the subroutine calls to this read-only memory.

2-3.4 A Blueprint for Writing Subroutines

We now offer a "blueprint" for calling and writing a subroutine, and a useful example of a subroutine. The following considerations should be given before anything is written:

1. What are the local and global variables, and what are the arguments and the results? How are the arguments and results to be passed: by value and result, by reference, or by name? Will they be passed in registers on the stack pointed to by S (or another stack), or through an argument list or an external list?
2. Will we use BSR or JSR calls, or should we use SWI calls?

When you write the subroutine call, the following sequence of code should appear (although sections that do not pertain may be omitted).

1. Push any arguments to be passed on the stack, and make "holes" for any results to be passed back on the stack. Set up the address of an external list, if it is used.
2. Jump to the subroutine.
3. Write the argument list, if it is used. Use FDB directives for addresses and 2-

word operands, FCB for 1-word operands, and FCC for character strings. These may be mixed in any order.

4. Pull any values returned on the stack and delete any values passed into the subroutine from the stack.

Inside the subroutine, the following sequences might be expected:

1. Make room on the stack for local variables. Initialize those local variables that may be read before data are written in them. Note that local variables can be simultaneously allocated and initialized by the PSH instructions.

2. Your routine can be put next.

a. If argument lists are used, set up an index register, such as Y, to point to the beginning of the list, as in TSY LDY 0,Y or by PULY, which pulls Y from the stack. The index register can be used for other operations, but must be reloaded with the argument list's address each time an argument is needed. If it is set up, a call by value 1-word argument can be put into accumulator A by the instruction LDAA n,Y, and a call by reference or name argument by LDX n,Y LDAA 0,X, where n is the argument's position in the list.

b. If a stack is used, arguments called by value on the stack can be picked up by TSX LDAA n,X, and call by reference or name by TSX LDX 0,X LDAA n,X, where n is the argument's position on the stack. (Remember: the return address is -usually- on the stack.)

Temporary variables can be pushed and pulled using PSH and PUL instructions, if they are pulled in the reverse order to which they are pushed, provided that the nested program segment rules are followed.

3. When you are ready to leave a subroutine, delete all local variables from the stack first. Use the return (RTS, PULX JMP n,X or RTI) that corresponds to the call. If an argument list is used, remember to move the program counter over the argument list before returning (several techniques for this have been shown.) If you should use the RTS or RTI to exit a subroutine but instead you use a BRA or JMP to jump to an error routine and do not return, remember to balance the stack by deleting the return address and other registers also saved on the stack.

2-3.5 Subroutine Documentation

We will introduce the need for clear documentation with a real example (adapted from a 6800 program to a 6811 program) that also conveys a very important additional concept, so we kill two birds with one stone. (Another reason for using a rather complex example will be given at the end of this discussion.)

This routine *hashes* a character string into an address. That is, it converts any string of characters you may type on a terminal into a number from 0 to 19, and that number is used as an address. The important hash routine property is that when you pass the same character string to it, you always get the same number to use as an address. The actual computations done inside the routine are not really important as long as they are

done the same way each time. Generally, the input characters are mixed together like restaurant "hash," hence the term. The routine is usually developed experimentally, though, to separate the character strings into different addresses as evenly as possible. Hashing is used to search for a character string to get information about it. Hashing helps narrow down the search by starting at a place indicated by the address provided by the routine, close to where the string starts. In particular, the hashing routine described soon is the actual routine that Motorola uses in its MDOS software to search a disc's directory. (In fact, if you know some of the terminology used with discs, this hash routine determines which sector of the directory the string is (probably) on. That sector is read, and the string is compared to all those strings on the sector. When a matching string is found, the track and sector number that begins the file is read from words after it.)

This subroutine needs a character string. If it is variable length (the Motorola routine fixes the length to ten), we won't want to pass the string by value but by reference. We can provide the address of the first character of the string. To account for the variable length, we can supply the length as a call by value parameter. The result is a small number, so it can be returned in a register. The parameters can be passed in an argument list to enhance storage efficiency. The length, which will go into an accumulator, and the address, which will go into an index register, can be picked up by a LDAA 0,Y instruction. To do this, the length is the first argument. The calling routine to hash a ten-character string at location ALPHA will then be written

```
BSR  HASH
FCB  10
FDB  ALPHA
```

The hash subroutine is shown next. Of importance to this discussion on subroutines, the first two lines pick up the arguments, and the last line returns past the argument list. The result is left in accumulator B.

HASH	PULY	GET RETURN ADDR., POINTS TO THE ARGUMENT
	LDAA 0,Y	GET STRING COUNT TO A
	LDX 1,Y	GET STRING ADDRESS TO X
	INY	MOVE RETURN ADDRESS PAST ARGUMENT LIST
	INY	
	INY	
	PSHY	PUT RETURN ADDRESS BACK ON THE STACK
	PSHA	SAVE COUNT STACK
	PSHX	SAVE STRING POINTER ON THE STACK
	CLRB	CLEAR THE PLACE TO COMPUTE THE RESULT
	TSY	Y BECOMES TOP-OF-STACK POINTER
LOOP	PSHB	SAVE RESULT AND CONDITION CODES ON STACK
	TPA	
	PSHA	SAVE CONDITION CODE
	LDX 0,Y	GET STRING POINTER
	LDAB 0,X	GET A CHARACTER
	INX	MOVE POINTER


```

STX 0,Y    PUT IT BACK ON THE STACK
SUBB #$25  MAKE CHARACTER UNIQUE
BPL NOCLR  IF NEGATIVE
CLR        MAKE IT ZERO
NOCLR PULA  RESTORE CONDITION CODES
TAP
TSX
ADCB 2,X   ADD TO MODIFIED CHARACTER
INS
ROLB       DOUBLE, FOLD CARRY BACK
DEC 2,Y    DECREMENT COUNT
BNE LOOP   UNTIL COUNT BECOMES ZERO, LOOP
RORB       UNDO LAST ROLB
LDAA #$11  MULTIPLY BY 11
MUL
ANDB #$1F  CLEAR HIGH THREE BITS
CMPB #19   IS IT WITHIN THE DESIRED RANGE?
BLS EXIT   IF SO, RESULT IS IN B
SUBB #20   PULL IT WITHIN RANGE
CMPB #9    IF NOW LOWER THAN 9
BHI EXIT   THEN DOUBLE IT
ASLB       AND THEN
BITB #2    EXTEND THE
BEQ EXIT   LOWEST BIT
ORAB #1
EXIT INS    DELETE COUNTER ON STACK
INS        DELETE POINTER ON STACK
INS
RTS        UPDATED RETURN ADDRESS IS ON STACK

```

You might wonder, when reading this subroutine, about some characteristics that affect the way you use it. These should be supplied as comments. Comments can be put on a full line in assembler language by beginning the line with a `**`. Comments for a subroutine should (really, must) have the following information:

1. A brief description of the subroutine's meaning that should enable readers to determine if this is the one they are looking for.
2. A list of all arguments, what they mean, and whether they are called by value and result, reference, or name.
3. A good clear example of a subroutine call, identifying which arguments are passed on the stack, through the argument list, or in registers, and identifying whether BSR or SWI calls are to be used.
4. A list of all other registers, global memory words, and temporary storage locations that are changed or destroyed by the subroutine.
5. A description of any error conditions and how they are reported, and any abnormal exits from the subroutine that may be taken to handle errors.

Finally, all subroutines, unless they would be worse with these features, should be position independent and reentrant. You should note in the comments at the beginning of the subroutine when either feature is lacking.

We cannot stress enough the need for good opening comments. With them, it is easy to know if, for example, the Y register is untouched by the subroutine, or how to supply an argument and where the result is to be found. Without them, the subroutine program has to be studied thoroughly each time it is used. That's usually too much trouble, as you can appreciate by the preceding example. Try to glean this critical information from the subroutine without the use of comments. (We deliberately chose this hash routine to make that point, because it is rather messy inside.)

The following is a good example of opening comments for the preceding subroutine:

```

*****
*
* HASH CONVERTS N LETTER CHARACTER STRING AT ADDRESS
*   ADDR INTO A NUMBER FROM 0-19
*
* CALLING SEQUENCE:
*
* BSR HASH
* FCB N CALL BY VALVE
* FDB ADDR CALL BY REFERENCE
*
* DESTROYS CC,A,X,AND Y
* RETURNS HASHED ADDRESS IN B

```

We have examined different strategies for calling subroutines and passing arguments, and different techniques for implementing those strategies. We should now be prepared to write subroutines for interface software.

2-4 Arithmetic Operations

A microcomputer provides some challenging problems in implementing arithmetic operations. An 8-bit word is really very short. Even a 16-bit number is often too small. Consider business data processing, for example. If we keep track of money in binary to the penny, an 8-bit unsigned number can keep track of only \$2.55, and a 16-bit unsigned number, only \$655.35. That's not enough for most applications. In 1946, von Neumann told us in his great paper that begot the computer, that scientific computers need at least 40 bits. IBM chose a 32-bit word for its 360/370 and has learned from experience that almost all scientific computations require double precision arithmetic (64 bits) because you do need about 40 bits for scientific computation. A 16-bit word is just fine for address calculations and for feedback control systems but is inadequate for just about everything else. So even if the 6811 has 16-bit arithmetic instructions, we have to study, as we do in this section, the effective handling of multiple precision arithmetic.